

UNIVERSITÉ PARIS CITÉ

CONDUITE DE PROJET / PROGRAMMATION ORIENTÉE
OBJET

Rapport de Projet

Implémentation en JAVA d'un jeu Worms-like

Groupe 13 - Équipe 7

Participants :

ARNAUD Hugo
MESNILDREY Valentin
NESI Romain
SAMBA Seth Ederik
SANE Souleymane

Année 2025 - 2026

Table des matières

1	Cahier des charges	3
2	Dépôt	3
2.1	Organisation des branches	3
2.2	Gestion des labels	4
2.3	Intégration Continue (CI/CD)	4
2.4	Règles de développement	5
2.5	Fichiers de documentation	5
3	Jalons	5
3.1	Sprint 0 : Initialisation du projet (18/10 - 27/10)	5
3.2	Sprint 1 : Version console minimale (27/10 - 7/11)	6
3.3	Sprint 2 : Version console complète (8/11 - 30/11)	6
3.4	Sprint 2b : Amélioration du dépôt (15/11 - 21/11)	7
3.5	Sprint 3 : Version graphique (1/12 - 28/12)	7
3.6	Sprint 4 : Dernière release (29/12 - 12/01)	8
4	Problèmes	9
4.1	Problèmes JAVA	9
4.2	Problèmes GitLab	11
4.3	Problèmes encore connus	11
5	Pistes d'extensions	11
5.1	Fusion d'armes	11
5.2	Progression entre manches	11
5.3	Mode de jeu	12
5.4	Jeu multijoueur	12
6	Architecture du projet	14
6.1	Représentation du modèle des classes	15
6.2	Plugins et dépendances	16
7	Sources	16

1 Cahier des charges

L’objectif du projet est d’implémenter en JAVA un jeu Worms-like, un jeu tour par tour où plusieurs équipes de vers s’affrontent sur un terrain destructible, avec une physique simple (gravité constante et vent optionnel), des collisions avec le sol et l’eau, et une élimination des vers à zéro point de vie ou en cas de chute dans l’eau.

Il repose sur le déplacement, les sauts et les tirs, avec un arsenal limité et une gestion des munitions. L’interface comprend un écran d’accueil, un HUD (Heads Up Display) affichant les informations essentielles, une jauge de temps et un inventaire commun. Un système de sauvegarde et de chargement en fichier texte est prévu, ainsi que des options de paramétrage. Le jeu proposera un mode solo contre un bot basique et un mode multijoueur local. Des fonctionnalités optionnelles seront ajoutées, comme des armes supplémentaires, de nouveaux objets et la génération de cartes. Techniquement, le projet est développé en Java SE 17 avec une architecture MVC, une vue console (TUI : Terminal User Interface) et une vue graphique (GUI : Graphique User Interface), des tests unitaires avec le framework JUnit, et une intégration continue via Gradle et GitLab CI.

Le rendu final inclura le dépôt GitLab avec les sources, la documentation (README, Javadoc, CHANGELOG, DESIGN, AUTHORS, LICENSE), un rapport PDF avec la structure du modèle des classes, un fichier de sauvegarde de partie ainsi qu’un fichier JAR exécutable. La méthodologie adoptée est Agile/Scrum, avec gestion des tâches et du code sur GitLab, workflow Git par branches et MR, et tests automatisés intégrés au pipeline CI/CD. Enfin, le projet respectera une organisation claire du code en packages (modèle, vue, contrôleur) et des ajouts graphiques et sonores simples (sprites, animations, bruitages, musique).

2 Dépôt

Le dépôt est accessible sur le serveur **Moule**¹ :

Nous avons mis en place une méthode pour faciliter le travail en équipe et le rendu, qui consiste en une organisation des branches, la création de labels ainsi qu’une intégration continue.

2.1 Organisation des branches

Le projet suit une structure de branches définie dans le fichier **CONTRIBUTING** :

- **main** : branche protégée contenant uniquement les versions stables (releases).
- **dev** : branche protégée et par défaut servant de point d’intégration pour les fonctionnalités validées.
- **feature** : branche protégée contenant toutes les nouvelles fonctionnalités.
- **bugfix** : branche protégée contenant des corrections de bug liés à la branche feature.
- **documentation** : branche protégée contenant des ajouts de Javadoc supplémentaires et les modifications des fichiers importants, comme README, CONTRIBUTING, CHANGELOG et DESIGN.

1. <https://moule.informatique.u-paris.fr/mesnldr/projet-worms>

- **tests** : branche protégée contenant des ajouts ou des modifications liés aux fichiers de test.

Nous avons aussi créé des branches intermédiaires qui ont été supprimées à la fin de chaque sprint. Ces branches avaient pour but de spécifier le travail effectué, et dans quel paquet.

2.2 Gestion des labels

Lors du développement du jeu, nous avons utilisé plusieurs **labels**², classés par priorité :

Étiquettes prioritaires :

- bug (bug à résoudre)
- URGENT (à traiter en priorité)
- à revoir (ticket ou MR incomplet et/ou à revoir)
- PRIORITAIRE (MR ou ticket le plus urgent)

Autres labels :

- compte rendu (compte rendu durant les différentes séances de TP)
- documentation (ajout ou modification de fichier(s) destiné(s) à la documentation)
- en cours (ticket attribué et démarré)
- en revue (attente de l'approbation d'au moins 2 relecteurs)
- fonctionnalité (ajout ou modification de fichier(s) pour la fonctionnalité du jeu)
- organisation (distribution de tâches)
- planning (planification de tâches)
- proposition (ticket ou MR en attente de validation de démarrage)
- structure (ajout ou suppression de dossiers)
- tests (ajout ou modification de fichiers de tests)

2.3 Intégration Continue (CI/CD)

Un pipeline d'intégration continue CI (configuré dans le fichier `.gitlab-ci.yml`) est exécuté à chaque modification sur le dépôt. Il assure :

- La compilation du projet avec Gradle
- L'exécution des tests unitaires (JUnit)
- La vérification du style de code (Checkstyle)
- La génération de la documentation (Javadoc)
- La release automatique sur la branche main

Cette automatisation garantit qu'aucun problème n'est intégré dans la branche de développement.

2. <https://moule.informatique.u-paris.fr/mesnldr/projet-worms/-/labels>

2.4 Règles de développement

Afin d'avoir un travail cohérent, nous avons adopté des règles détaillées dans le fichier CONTRIBUTING :

- **Nommage des commits** : chaque commit suit la forme "type de contribution (fichier(s) ou paquet(s)) : description des changements". Par exemple, un message de commit pour la modification du dossier rapport : "modif(rapport/) : ajout de la section dépôt".
- **Processus de validation** : toute modification passe par une **MR**. Pour être fusionnée, une MR doit :
 - Être approuvée par au moins deux relecteurs (sauf cas particulier).
 - Valider l'ensemble du pipeline CI (compilation, tests, checkstyle, documentation).
 - Ne contenir aucun conflit avec la branche cible.

2.5 Fichiers de documentation

De plus, notre dépôt contient plusieurs fichiers de documentation :

- AUTHORS (renseigne tous les participants du projet)
- CHANGELOG (listes de toutes les implémentations)
- CONTRIBUTING (manière de contribuer au projet)
- DESIGN (raisons des implémentations)
- LICENSE (définition des droits utilisateurs)
- README (explications du fonctionnement de lancement du jeu)
- SPECIFICATIONS (cahier des charges)

Ces fichiers de documentation sont principalement destinés à des personnes extérieures.

3 Jalons

3.1 Sprint 0 : Initialisation du projet (18/10 - 27/10)

- [Lien](#) vers le sprint 0.

Objectifs :

Mettre en place l'environnement de travail et l'organisation du projet afin que nous soyons prêts à commencer le développement du jeu.

Le but est de préparer tout ce qui est nécessaire : GitLab, workflow, planification, etc.

Durant ce premier sprint, nous avons concentré nos efforts sur l'initialisation du dépôt GitLab et la structuration du projet. Cela comprend la mise en place des pipelines CI/CD (.gitlab-ci.yml), ainsi que la rédaction des fichiers essentiels : README, LICENSE, AUTHORS, CONTRIBUTING, CHANGELOG.

Chaque tâche a été intégrée au dépôt via un ticket attribué et une MR validée par au moins 2 personnes.

L'objectif de ce sprint n'était pas d'avoir dès le début du code, mais de préparer la suite du développement du jeu : définition du workflow Git, structuration des branches, configuration des outils de CI/CD et mise en place des supports de communication et de planification (ticket, tableau des tickets, jalons).

3.2 Sprint 1 : Version console minimale (27/10 - 7/11)

— [Lien](#) vers le sprint 1.

Objectifs :

Création de l'architecture

Initialisation des bases (fichiers, constantes)

Version console minimale

Durant ce sprint, chaque fonctionnalité a été développée dans une branche dédiée, puis intégrée via une MR. Les classes essentielles (Guns, Worm, Main, Team, Game) ont été définies dès le début et réparties entre les membres du groupe. Elles ont été accompagnées de tests unitaires réalisés avec JUnit 5 pour faciliter la détection d'erreurs.

La gestion des bugs et les ajustements structurels ont été faits tout au long du jalon, permettant de corriger rapidement les problèmes rencontrés et d'améliorer la cohérence de l'architecture. Chaque fonctionnalité a fait l'objet d'un ticket, suivi d'une ou plusieurs MR.

En résumé, ce sprint a permis de poser les bases du projet : une bonne architecture, des classes essentielles déjà testées et une première version console jouable. Le code JAVA de ce jalon représente une bonne base pour les sprints suivants, où les fonctionnalités seront enrichies sans remettre en cause cette structure.

3.3 Sprint 2 : Version console complète (8/11 - 30/11)

— [Lien](#) vers le sprint 2.

Objectifs :

Ajout de sauvegarde/chargement d'une partie

Vue textuelle 'ASCII'

Changement de la structure pour adopter une MVC

Ajout d'une classe Inventory et Tools

Implémentation des méthodes pour se déplacer dans la vue ASCII et réimplémentation des méthodes de tirs

Dans la continuité du sprint 1, les efforts se sont majoritairement concentrés sur l'amélioration des fonctionnalités du jeu et la restructuration de l'architecture (avec

le sprint 2b créé en parallèle) afin de la rendre plus claire, maintenable et évolutive. Le changement vers un modèle MVC a constitué une étape importante du sprint, car il a nécessité une restructuration totale des classes existantes et une séparation claire entre modèle, vue et contrôleur. De plus, cette refonte a permis d'améliorer la lisibilité du projet.

De plus, une vue ASCII a été implémentée afin d'avoir une représentation plus claire du jeu (carte et worms). La gestion de la sauvegarde et du chargement de partie a également été ajoutée.

En résumé, ce sprint a marqué la fin de la version console avec une meilleure architecture (cf. sprint 2b) et une vue ASCII fonctionnelle.

3.4 Sprint 2b : Amélioration du dépôt (15/11 - 21/11)

— [Lien](#) vers le sprint 2b.

Ce sprint est un sprint parallèle, visant à améliorer le dépôt sans se mélanger avec le sprint de la version console (sprint 2).

Objectifs :

Rendre visible la Javadoc

Pouvoir exécuter le jeu avec des arguments en utilisant Gradle (et non les scripts)

Durant ce sprint 2b, le dépôt a été optimisé pour améliorer la navigation. Nous avons configuré la génération automatique de la Javadoc via Gradle, rendant la documentation du code accessible et automatique. Parallèlement, nous avons modifié la configuration de Gradle pour permettre le passage d'arguments à l'application lors de l'exécution (via le flag `-args`), permettant de lancer le jeu dans différents modes (console, développeur) directement depuis l'outil de build, indépendamment des scripts systèmes.

3.5 Sprint 3 : Version graphique (1/12 - 28/12)

— [Lien](#) vers le sprint 3.

Objectifs :

Ajout de toute la partie graphique (AWT et Swing)

Destruction du terrain

Utilisation du clavier et de la souris pour les contrôles

Déplacement horizontal de la carte + zoom (optionnel)

Ajouter un bot (optionnel pour ce jalon)

Actualisation des fichiers README, DESIGN, Rapport

Pendant ce sprint, l'objectif était de faire la transition du projet d'une version console (TUI) à une version graphique (GUI). L'interface utilisateur a été conçue en utilisant les bibliothèques AWT et Swing, permettant ainsi l'affichage des graphismes pour la carte, les worms ainsi que les interactions utilisateurs (bouger, viser, tirer) avec le clavier et la souris. Pour faire cela, une modification de l'architecture existante a été faite pour garantir une interaction entre la logique du jeu (modèle) et l'interface utilisateur (vue).

Un bot a aussi été inclus pour offrir un adversaire au joueur solo. Ce bot utilise des règles simples, permettant d'essayer le jeu en solo et de vérifier son bon fonctionnement. Il n'est pas encore totalement performant à ce niveau, mais il offre néanmoins une meilleure expérience de jeu.

Comme les précédents sprints, chaque fonctionnalité a été conçue dans une branche spécifique et ajoutée grâce à des MR. Les documents (README, DESIGN, Rapport) ont aussi été actualisés pour mettre à jour l'avancement du projet.

3.6 Sprint 4 : Dernière release (29/12 - 12/01)

— [Lien](#) vers le sprint 4.

Ce sprint a eu différents objectifs, que nous avons classés en fonction de leur priorité.

URGENTS :

- ☒ Ajouter une caméra avec un zoom et un scroll horizontal
- ☒ Permettre la possibilité de passer le tour en version GUI

MOINS URGENTS :

- ☒ Ajouter des dégâts de zone avec certaines armes
- ☒ Implémentation d'un bot fonctionnel (avec une difficulté réglable)
- ☒ Lisser les déplacements des worms au lieu d'un déplacement case par case
- ☒ Permettre une génération de cartes variées en fonction d'un thème
- ☒ Ajouter d'autres armes (ricochet, grenade à retardement, mine, tourelle, corps à corps...)
- ☒ Ajouter une option de vent sur le terrain afin d'influencer la trajectoire des projectiles

OPTIONNELS :

- ☒ Ajout d'autres outils (grappin, jet-pack, foreuse)
- ☒ Ajout des paramètres avancés (ajuster la durée du tour, PV initiaux, munitions, dégâts de chute, friendly fire)
- ☐ Progression entre manches / écran d'achat d'objets entre parties
- ☐ Fabrication d'armes (fusion d'objets)
- ☒ Améliorer les graphismes
- ☒ Ajouter des sons
- ☒ Ajouter des animations

Ce dernier sprint visait à finaliser le projet en remplissant tout ce qui avait été inclus dans le cahier des charges. Pour cela, nous avons classé les tâches par ordre d'importance afin d'assurer un jeu complet à temps.

Les tâches urgentes, comme l'ajout de la camera avec zoom et de déplacements horizontaux et la capacité de passer le tour en version graphique ont été implémentés assez rapidement.

Pour les tâches moins urgentes, comme les dégâts de zone pour certaines armes, ont été implémentées en parallèle. Le bot du sprint 3 a été amélioré avec un niveau de difficulté réglable et de meilleurs décisions (déplacements, visée très précise, etc.). Un lissage des mouvements des vers, une variation des cartes selon un thème choisi, de nouvelles armes, etc. ont suivi pour rendre le jeu plus riche et plus complet.

Finalement, pour ce qui est des options, nous avons ajouté plus de réglages dans le SettingsPanel, comme choisir la durée du tour, l'activation du friendly fire, le vent, l'affichage du nom des équipes, le choix de la carte, etc. Nous avons aussi amélioré les graphismes du jeu (tuiles et sprites), les sons (musique et bruitage) et ajouté des particules pour avoir quelques animations.

Malheureusement certains points n'ont pas été réalisés, comme la continuité entre les "sessions de jeu", une boutique pour acheter de nouvelles armes ou encore la création d'armes. Pendant cette phase, nous avons surtout amélioré les derniers petits détails, notamment en corrigeant les bugs, mais plus généralement en améliorant la qualité globale du jeu et l'expérience utilisateur (menu et gameplay). Encore une fois, chaque ajout est passé par des MR, vérifiées par au moins 2 relecteurs, garantissant ainsi une version finale correcte.

Finalement, ce dernier jalon nous a permis de livrer un jeu fini et abouti avec la plupart de nos idées. Même si nous avons manqué de temps pour certaines fonctionnalités que nous aurions voulu implémenter, le cahier des charges est respecté. Nous avons réussi à développer un jeu immersif dans lequel le joueur peut jouer sans bug (connu) et de manière fluide et agréable. Les réglages donnent plus de choix et apportent une personnalisation des parties. Les contraintes et attentes du début du projet sont respectées et atteintes.

4 Problèmes

Tout au long du développement du jeu, nous avons fait face à quelques bugs, dont certains ont été plus compliqués à résoudre que d'autres. Certains ont touché la partie programmation, avec le code JAVA, tandis que d'autres ont concerné le workflow GitLab.

4.1 Problèmes JAVA

Lors de notre développement de la partie console (jalons 1 à 2b), aucun bug remarquable (cassant le jeu) n'était à déclarer. Cela concernait surtout le manque de méthode pour subvenir à un besoin, ou des problèmes mineurs lors de la mise en place du système

de sauvegarde. Lors du développement de la partie graphique (jalons 3 à 4), plusieurs bugs sont apparus :

- TurnManager : l'implémentation du système de tour en version graphique ne fonctionnait pas correctement.
- Worm fantôme : un worm qui tirait alors qu'il était dans les airs (saut ou chute) disparaissait.
- Inventaires : les inventaires n'étaient pas communs en version console, mais le sont devenus dans la version graphique (liée à la gestion des tours en graphique qui ne mettait pas à jour l'inventaire).
- Version console : le développement de la partie graphique a rendu impossible l'exécution pourtant fonctionnelle de la version console.
- Carte : selon les dimensions des écrans de chacun, la carte ne s'affichait pas en entier et le scrolling horizontal n'était pas possible.
- Plein écran : selon la machine de chacun et avec un code identique, la fenêtre principale (JFrame) ne se mettait pas en plein écran pour tout le monde, et pas avec les mêmes dimensions.
- Pas de sons dans le JAR : la façon de lire les sons était incorrecte.
- Problèmes de déplacements : lorsque qu'un worm se déplace, il détruit les décorations. De plus, il peut se déplacer sur l'eau sur la carte Bridge.
- Problème de saut : notamment sur la carte Bridge, un worm ne pouvait pas sauter, alors même qu'il avait l'espace nécessaire et qu'il n'était ni en l'air ni sur l'eau.
- Problème de zoom : les crates (caisses de ravitaillement) n'étaient pas affectées par le zoom. Cela avait pour effet de les rendre très petites à côté des tuiles du terrain.
- Problème de tir : si 2 worms sont trop proches, le tir de l'un ne touchera pas l'autre, même s'ils sont dans des équipes différentes.
- Problème de génération de la carte Bridge : sur la carte Bridge, de l'eau apparaissait sur le pont.
- Problème de spawn : sur la carte Bridge, un worm peut spawn en dehors de la carte.

Certains de ces problèmes ont été résolus en supprimant ou en changeant une petite partie de la logique de TurnManager.

Pour résoudre les problèmes visuels, comme l'affichage de la carte et l'affichage en plein écran, nous avons remplacé nos variables fixes qui définissent les dimensions de l'écran en des variables dynamiques. Un problème graphique qui persiste encore est l'affichage en plein écran, car aucun compromis n'a été trouvé pour résoudre ce problème.

Pour les problèmes sonores, il a suffi de modifier le chemin vers les sons ainsi que la méthode qui permet d'avoir un chemin relatif. Enfin, les problèmes de déplacements, saut, zoom, tir, génération de la carte, spawn ont été les derniers problèmes résolus. En ce qui concerne le saut, il manquait l'assignation d'une variable qui était nulle par défaut. Pour le problème de la génération de la carte Bridge, la suppression d'une partie du code de la fonction generateBridgeMap() de la classe Map a résolu ce problème. Pour résoudre le problème du worm qui détruit les décorations quand il se déplace, il a suffi de supprimer la classe interne CellBackup de la classe Worm, ainsi que la suppression des modifications de cellules dans la méthode update(Map, GuiView). Pour le spawn, la fonction findValidePosition(Map) de la classe Team a été modifiée en décrémentant la valeur de la largeur de la carte. Le zoom des crates a été résolu en supprimant 2 attributs

qui définissaient la taille des crates et en modifiant la méthode `renderCrate(Graphics, Crate, int)` de la classe `CrateRenderer` pour remplacer ces 2 variables.

4.2 Problèmes GitLab

Du côté de GitLab, 3 problèmes sont très vite apparus durant les 2 premiers sprints :

- Release : lors de la 2e release, la branche dev était fusionnée dans la branche main, mais n'était donc plus à jour avec cette dernière (commit de retard).
- Branches : avec seulement des branches protégées et sans branches intermédiaires, des problèmes de commits d'avance ou de retard sont apparus.
- Pipelines : les pipelines étaient de plus en plus longues au fil de l'avancement du projet.

Afin de ne pas avoir de problèmes lors des releases suivantes, des MR systématiques de main vers dev, feature, documentation, bugfix et tests ont été effectuées. En clair, toutes les branches protégées ont été mise à jour avec main.

Toutes les branches intermédiaires (non protégées) ont été supprimées à la fin de chaque jalons afin d'éviter tout problème de version (commits d'avance ou de retard).

La création de 2 runners personnels a permis de ne pas dépendre du runner principal, et ainsi augmenter la productivité. Il a aussi fallu modifier le script CI afin de prendre en compte ou non le proxy, car les runners personnels n'ont pas besoin du proxy mis en place par l'université.

- Runner de Hugo³
- Runner de Valentin⁴

4.3 Problèmes encore connus

5 Pistes d'extensions

Au terme du développement de notre jeu, plusieurs pistes d'extensions ont été envisagées.

5.1 Fusion d'armes

Un système d'artisanat (crafting) offrirait la possibilité de combiner des armes pour en obtenir d'autres plus puissantes ou obtenir des effets différents. Cela enrichirait la stratégie de gestion de l'inventaire et la collecte des caisses de ressources sur la carte. Pour faire cela, un menu `CraftingPanel` étendant `JPanel` (accessible depuis une touche du clavier ou un bouton à l'écran) peut être mis en place où le joueur utiliserait le Drag and Drop pour fusionner deux de ses armes et ainsi en obtenir une nouvelle.

5.2 Progression entre manches

L'introduction d'un système d'argent entre les parties permettrait une progression sur le long terme. Les joueurs pourraient gagner des crédits en jouant pour débloquent de

3. <https://moule.informatique.u-paris.fr/mesnildr/projet-worms/-/runners/116>

4. <https://moule.informatique.u-paris.fr/mesnildr/projet-worms/-/runners/121>

nouvelles armes avant le début d'une partie ou recharger toutes les munitions d'une arme en pleine partie, ou améliorer les statistiques de leurs vers, comme les points de vie, la vitesse et la résistance aux dégâts de chute. Un menu ShopPanel étendant JPanel sera rajouté à l'écran de fin pour donner la possibilité au joueur d'acheter des munitions ou des armes. Dans ce sens, le joueur aurait sa propre équipe qu'il améliore au fil du temps. La complexité serait de donner au bot la meilleure décision à prendre entre toutes les options de ce menu.

5.3 Mode de jeu

Au-delà du match à mort par équipe, des modes alternatifs, comme le 'Battle Royal' (seul contre tous) ou le 'Zombie' (un worm qui meurt passe dans l'équipe qui l'a éliminé) renouvelleraient l'intérêt du jeu et exploiteraient différemment les mécaniques de déplacement et de combat. Une option dans le SettingsPanel permettrait de sélectionner le mode de jeu voulu. Dans le mode 'Zombie', la méthode qui permet de supprimer un vers du jeu sera adapté pour le faire changer d'équipe et remettre ses points de vie au maximum. Cette méthode modifiera aussi son apparence (skin).

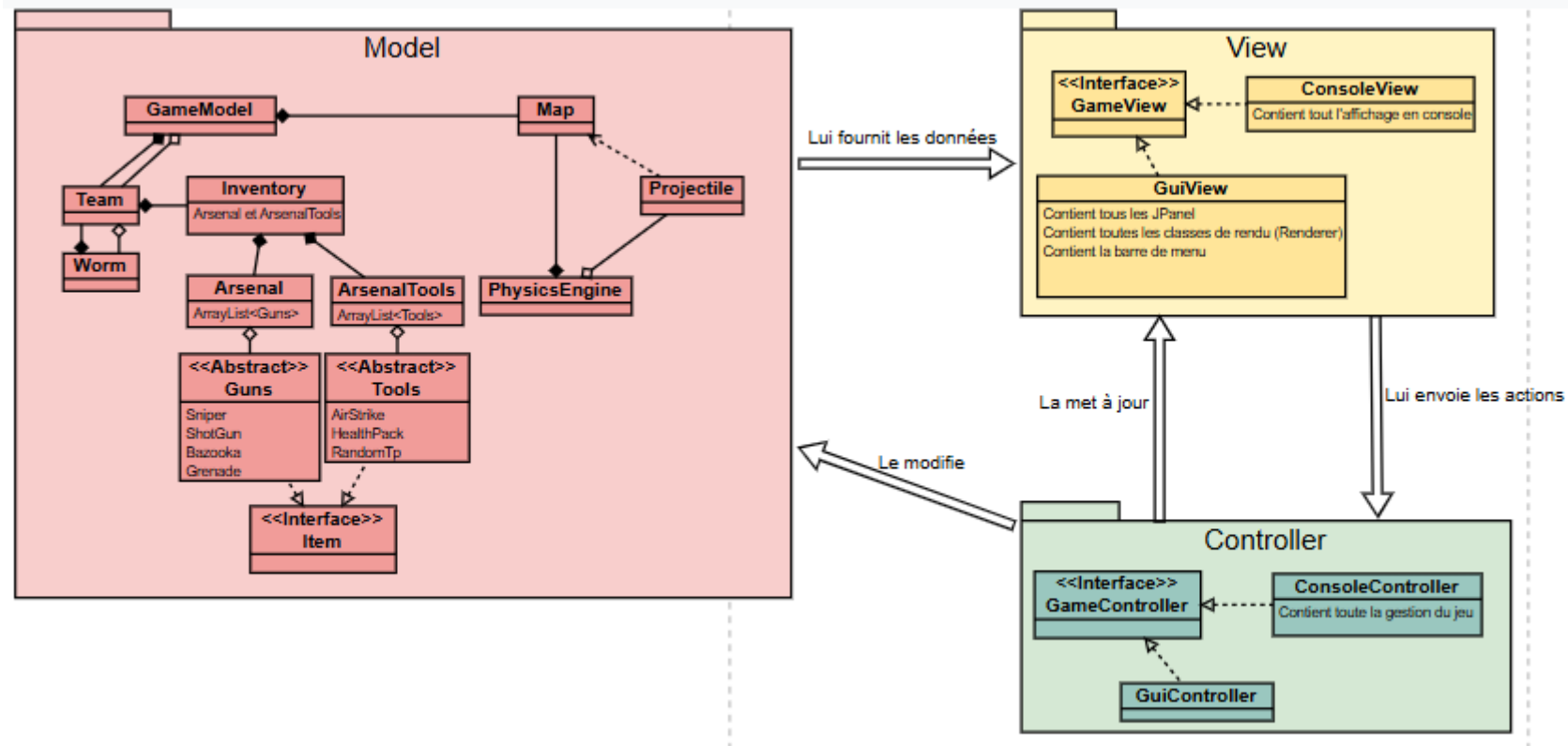
5.4 Jeu multijoueur

Tout d'abord, l'ajout d'une fonctionnalité réseau permettrait à plusieurs joueurs de jouer à distance, chacun avec sa propre équipe. En hébergeant une partie sur sa propre machine, un des joueurs jouerait le rôle de "serveur" et gérerait l'état du jeu, la physique et le rendu, tandis que les joueurs "clients" y enverraient leurs actions. Ensuite, la création d'un menu spécifique MultiplayerPanel (accessible depuis le menu principal) donnerait au joueur la possibilité de choisir entre héberger une partie (JButton) ou en rejoindre une (liste de parties disponibles dans un JScrollPane et un JButton pour confirmer le choix). Enfin, ces pistes combinées avec les précédentes rendraient le jeu encore meilleur avec différentes manières de jouer.

6 Architecture du projet

src/main/java/			
app	controller	model	view
Launcher.java Main.java	bot BotAction.java BotController.java BotDecisionEngine.java CollectCrateAction.java MoveAction.java ShootAction.java UseToolAction.java console ConsoleController.java InputValidator.java gui AimController.java GuiController.java InputController.java ShootController.java GameController.java GameInitializationService.java	items crates Crate.java CrateContentType.java CrateManager.java guns Bazooka.java Grenade.java Guns.java ShotGun.java Sniper.java tools AirStrike.java HealthPack.java RandomTp.java Tools.java Arsenal.java ArsenalTools.java Inventory.java Item.java persistence LoadManager.java SaveManager.java physics Projectile.java TrajectoryPredictor.java Wind.java players Team.java Worm.java Config.java GameModel.java Map.java	console AnsiColor.java ConsoleHelper.java ConsoleView.java gui CrateRenderer.java DevModRenderer.java EndGamePanel.java GameMenu.java GamePanel.java GuiView.java HomePanel.java InventoryPanel.java MapRenderer.java Particle.java SavePanel.java SettingsPanel.java SoundPlayer.java TrajectoryRenderer.java WindRenderer.java WormRenderer.java GameView.java

6.1 Représentation du modèle des classes



6.2 Plugins et dépendances

La section suivante contient la liste de tous les plugins et dépendances utilisés dans notre projet pour Gradle et le code JAVA.

- java : activation du plugin Java de Gradle
- application : permet de définir un mainClass et la tâche run
- checkstyle⁵ : vérification de style
- jacoco : activation de l’outil pour la couverture de code
- com.github.ksoichiro.console.reporter⁶ : utilisation du plugin pour afficher les rapports de tests dans la console
- JUnit 5 : utilisation de l’api⁷ et d’engine⁸ pour faire des tests
- Mockito⁹ : simulation des interfaces
- AWT : interface graphique bas niveau
- Swing : interface graphique composants

7 Sources

Pour les armes et outils, nous avons utilisé les images du site **Flaticon**¹⁰. Les bruitages proviennent du site **ElevenLabs**¹¹. Enfin, les musiques ont été réalisées par Valentin.

5. <https://mvnrepository.com/artifact/com.puppcrawl.tools/checkstyle/10.13.0>

6. <https://github.com/ksoichiro/gradle-console-reporter>

7. <https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-api/5.11.0>

8. <https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-engine/5.11.0>

9. <https://mvnrepository.com/artifact/org.mockito/mockito-core/5.11.0>

10. <https://www.flaticon.com/fr/>

11. <https://elevenlabs.io/fr/>